COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 20, 2026	**DATE SUBMITTED:**	Oct 20, 2026
NAME:	Denzen Igloso	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 14.1 ALGORITHMIC COMPLEXITY

1. PROCEDURE OUTPUT

	Best Case	Worst Case
Insertion sort	O(n)	O(n^2)
Selection sort	O(n^2)	O(n^2)
Igloso (Analysis)	The best case for insertion sort is that the array was already sorted and the next element to be sorted was already in the right places so nothing swaps. While in the selection sort, it always scans all of the elements in the array no matter how the elements is arranged, the only thing that change is the number of switches.	The worst case for insertion sort is that the array is in the reverse order, meaning it will compared all the elements until the array become sorted. While in the selection sort, it is the same as best case but the only difference is the number of switches along the array.

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

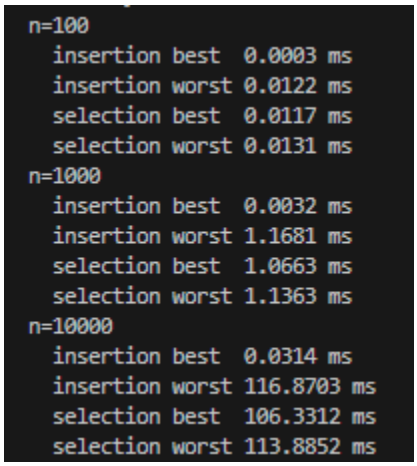
	Insertion sort and Selection sort
Best Case	
Worst Case	

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1,000	0.0233 us	0.9684 us
10,000	0.0304 us	9.1111 us
100,000	0.0363 us	94.3241 us

act14.cpp > binarySearch(const int [], int, int)

C:\Emptech\IGLOSO\act14.cpp

```cpp
#include <chrono>
#include <vector>
#include <cstdio>

int linearSearch(const int arr[], int n, int target) {
    for (int i = 0; i < n; i++)
        if (arr[i] == target) return i;
    return -1;
}

int binarySearch(const int arr[], int n, int target) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

volatile int sink;
```

```cpp
template <typename F>
double timeSearch(F f, const std::vector<int>& a, int target, long reps) {
    auto t0 = std::chrono::high_resolution_clock::now();
    for (long r = 0; r < reps; r++) sink = f(a.data(), (int)a.size(), target);
    auto t1 = std::chrono::high_resolution_clock::now();
    return std::chrono::duration<double, std::micro>(t1 - t0).count() / reps;
}

int main() {
    int sizes[] = {1000, 10000, 100000};
    for (int n : sizes) {
        std::vector<int> a(n);
        for (int i = 0; i < n; i++) a[i] = i * 2;
        int target = -1;
        long reps = 20000;
        double b = timeSearch(binarySearch, a, target, reps);
        double l = timeSearch(linearSearch, a, target, reps);
        printf("N=%d  binary=%.4f us  linear=%.4f us\n", n, b, l);
    }
}
```

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\msys64\ucrt64\bin> C:\Emptech\IGLOSO\act14.exe
N=1000  binary=0.0233 us  linear=0.9684 us
N=10000  binary=0.0304 us  linear=9.1111 us
N=100000  binary=0.0363 us  linear=94.3241 us
PS C:\msys64\ucrt64\bin> (Set-ExecutionPolicy -Scope Process
GLOSO\PyQt6\Scripts\Activate.ps1)
(PyQt6) PS C:\msys64\ucrt64\bin>
```

Analysis (IGLOSO)

- Based on the results, the linear search takes longer as the input size gets bigger. Every time N increases by 10 times, the time also increases by about 10 times (0.9684 to 9.1111 to 94.3241 µs), so it is O(n). The binary search stays almost the same (0.0233 to 0.0363 µs) even when N became 100,000, because it only cuts the array in half every step, so it is O(log n). At 100,000 elements, binary search is about 2,600 times faster than linear search, but the only catch is that the array must be sorted first, while linear search can work on unsorted data.

Conclusion

- In conclusion, I learned that binary search is faster than linear search when the data is big. In the experiment, the time of linear search became longer when the input size got bigger, but the time of

binary search almost stayed the same. The only problem with binary search is that the data must be sorted first, while linear search can be used even if it is not sorted.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used grammarly to fix my grammar mistakes and provide good constructed words for a better easy understanding.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.